



Middleboxes, which provide layer 3-7 network services, are a crucial part of many enterprise data centers and cloud deployments. They help ensure security, improve performance, and meet other sophisticated goals.

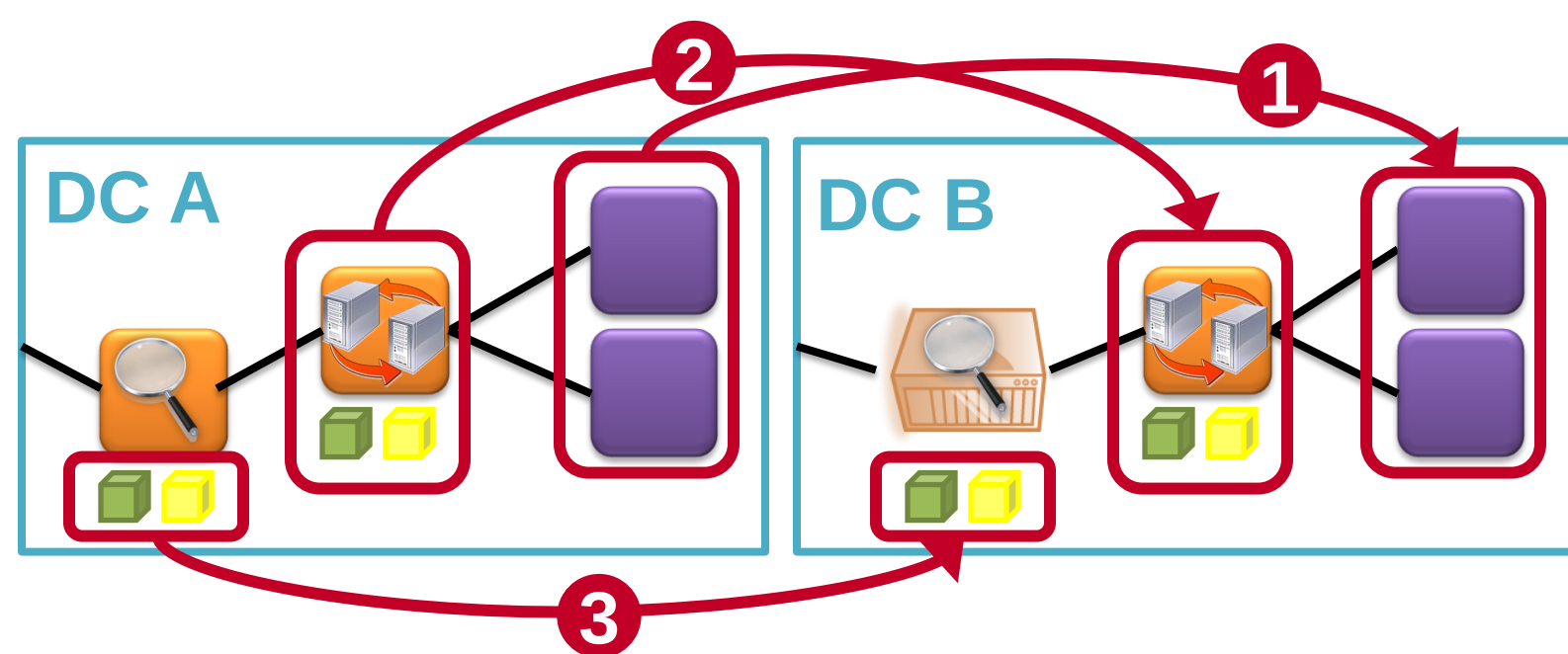
Traditionally, enterprise data centers have included dedicated physical appliances at network chokepoints. However, middleboxes are now being deployed in a variety of additional forms—as VMs, in hypervisors, or as collections of processes—and placed at arbitrary points in the network.

Unfortunately, current approaches for managing middleboxes are limited and clumsy, reducing enterprises' abilities to create more dynamic deployments. We address this issue by designing a *software-defined middlebox networking* framework that simplifies management and engenders rich, new applications.

Middlebox Deployments in Data Centers

Motivating Scenarios

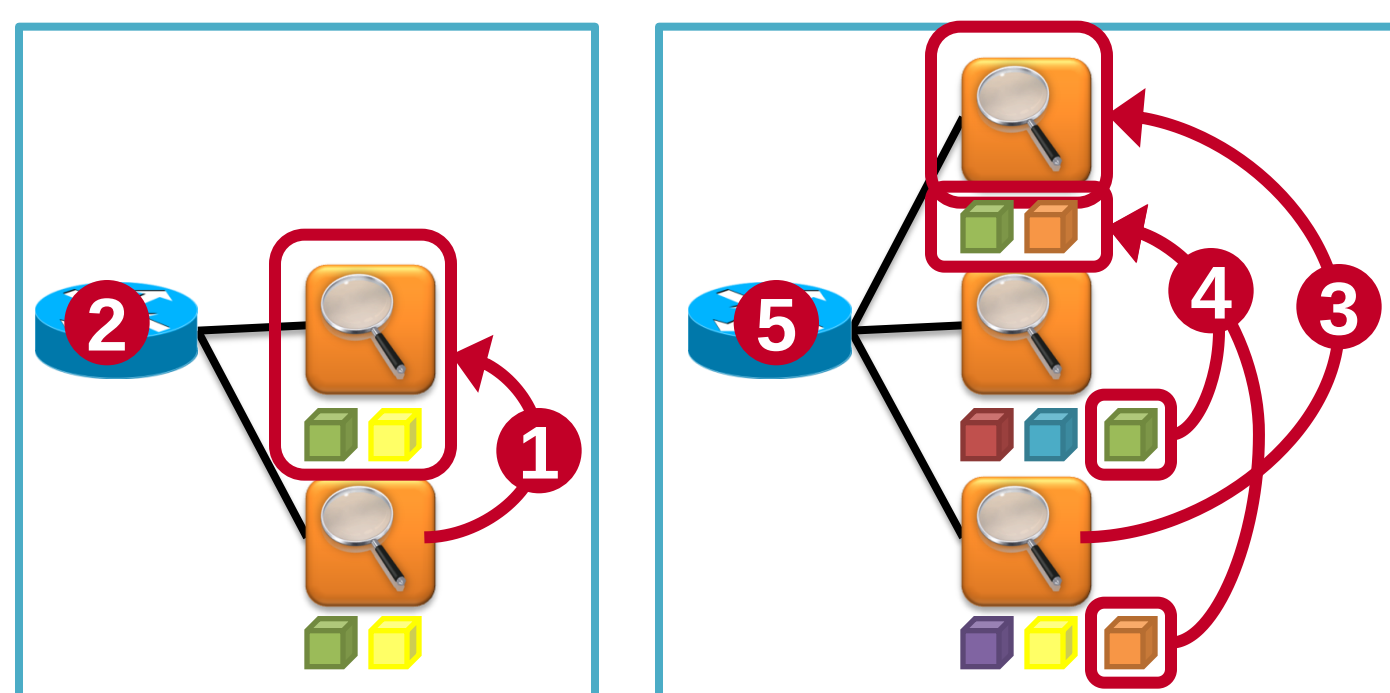
Live migration between software-defined data centers



- 1 & 2) Migration of application VMs & middlebox VMs
- 3) Transfer of middlebox state to physical appliance

Requires programmatic control over middlebox state

Middlebox scaling and load balancing

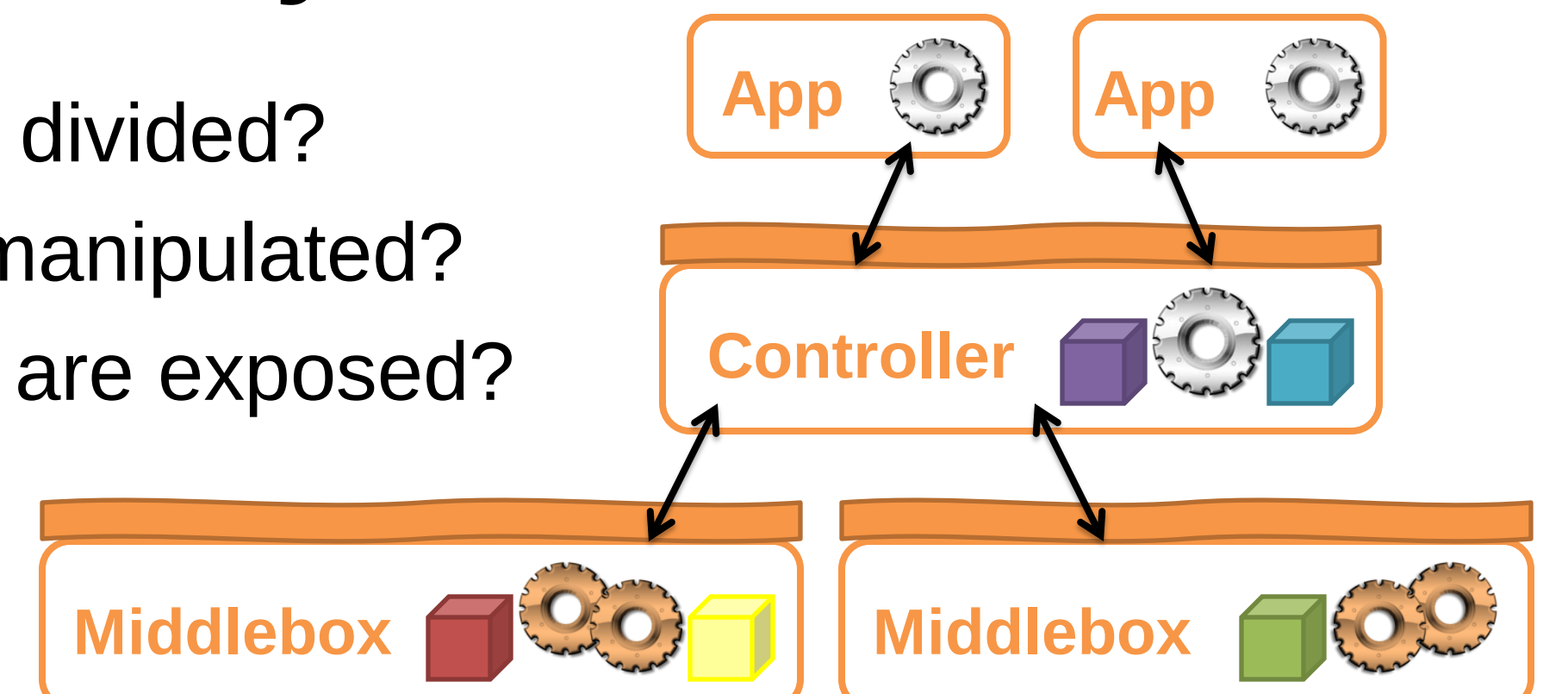


- 1) Clone middlebox VM
- 2) Divide flows evenly
- 3) Clone middlebox VM
- 4) Re-balance flows by moving per-flow state
- 5) Divide flows evenly

Requires fine-grain control over middlebox state

Key Issues

- 1) How is the logic divided?
- 2) Where is state manipulated?
- 3) What interfaces are exposed?

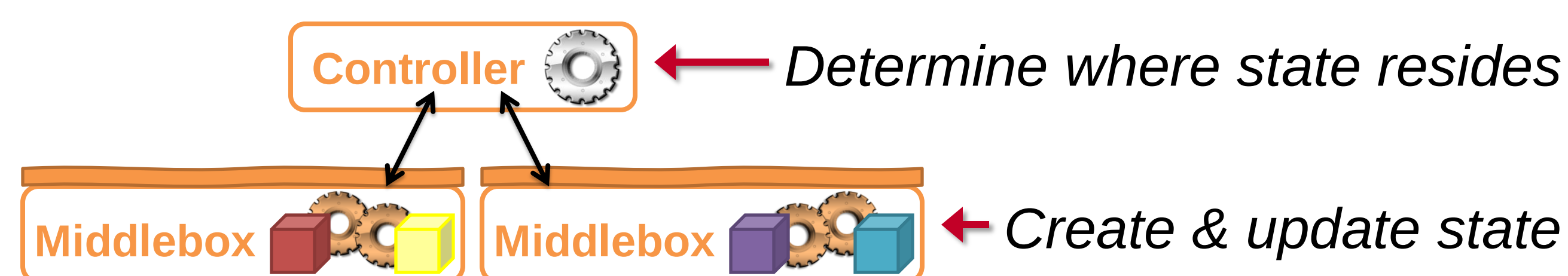


Classification of Middlebox State

Class	Definition	
Action	Defines operations to apply to packets/flows	} Few forms
Supporting	Helps decide between possible actions	
Tuning	Tunes algorithms for performance/efficiency	} Does not affect correctness

Manage out-of-band

Manipulating State



Broad Operations Interface

get (**Filter**
 SrcIP = 10.10.54.41) → **Key**
 SrcIP = 10.10.0.0/16
 DPort = 22 **Action**
 *

add (**Key**
 DstIP = 10.20.1.0/24 **Action**
 DROP) →

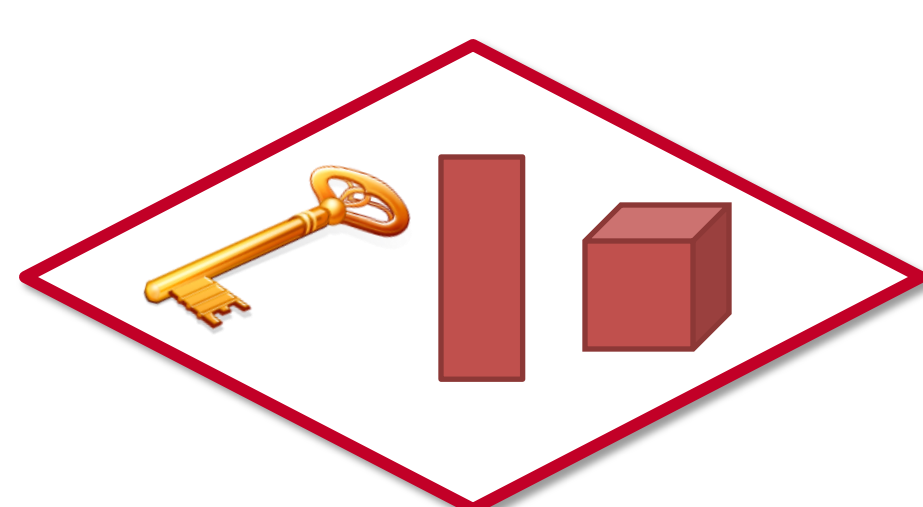
Src	Dst	Proto	Action
*	10.20.1.0/24	TCP	DROP

remove (**Filter**
 ...)

Expose State-Related Events

Triggers

- 1) Created/updated state
- 2) Require state to complete operation



Contains

- Key
- Packet
- State

Representing State

Key	Action	Supporting
Field1 = Value1 ... FieldN = ValueN	Offset1 → Const1 ... OffsetN → ConstN	Binary Blob
<i>Identify subset of traffic to which state applies based on protocol header fields</i>	<i>Encode most actions of common middleboxes using transformation functions</i>	<i>Structure depends on deep internal middlebox logic</i>

Open Questions

- 1) Representing shared (i.e., not per-flow) state
- 2) Encoding supporting state & other action state
- 3) Preventing invalid state manipulations
- 4) Maintaining operation during state changes
- 5) Design of control logics for a variety of scenarios

